

DOM Traversal Practice Questions

HTML Structure (Use for all questions)

```
<div id="grandparent" class="container">
  <div id="parent1" class="box">
    <p id="child1" class="text">Hello</p>
    <p id="child2" class="text">World</p>
    <span id="child3">!</span>
  </div>
  <div id="parent2" class="box">
    <ul id="list">
      <li class="item active">One</li>
      <li class="item">Two</li>
      <li class="item active">Three</li>
    </ul>
  </div>
</div>
```

Q1: Parent & ParentElement

Task: Get the parent of `#child1` , then get the grandparent from there.

Solution:

```
const child = document.getElementById('child1');
child.parentElement;           // #parent1
child.parentElement.parentElement; // #grandparent
```

Q2: closest() — Ancestor Search

Task: From the first `<li>` , find the nearest ancestor with class `container` .

Solution:

```
const li = document.querySelector('li');
```

```
li.closest('.container'); // #grandparent
// closest() walks UP the tree and returns first match
// li.closest('.box') → #parent2
```

Q3: children vs childNodes

Task: Get children of `#parent1`. What's the difference?

Solution:

```
const p = document.getElementById('parent1');
p.children;    // HTMLCollection [p, p, span] – elements only
p.childNodes;  // NodeList [text, p, text, p, text, span, text] – includes whites
```

Key: `children` = elements only, `childNodes` = all nodes including text/comments.

Q4: firstElementChild / lastElementChild

Task: Get first and last element child of `#parent1`.

Solution:

```
const p = document.getElementById('parent1');
p.firstElementChild; // <p id="child1">
p.lastElementChild;  // <span id="child3">

// vs firstChild/lastChild which may return text nodes
p.firstChild; // #text (whitespace)
```

Q5: Sibling Navigation

Task: Starting from `#child2`, get previous and next sibling elements.

Solution:

```
const c2 = document.getElementById('child2');
c2.previousElementSibling; // <p id="child1">
c2.nextElementSibling;     // <span id="child3">
```

```
// previousSibling/nextSibling return text nodes too
c2.previousSibling; // #text (whitespace)
```

Q6: Walk Up — Count All Ancestors

Task: Write code to collect all ancestors of the first `<li>` until `document`.

Solution:

```
function getAncestors(el) {
  const ancestors = [];
  let current = el.parentElement;
  while (current) {
    ancestors.push(current.tagName + (current.id ? '#' + current.id : ''));
    current = current.parentElement;
  }
  return ancestors;
}

getAncestors(document.querySelector('li'));
// ["UL#list", "DIV#parent2", "DIV#grandparent", "BODY", "HTML"]
```

Q7: contains() & Descendant Check

Task: Check if `#grandparent` contains the first `<li>`. Check if `#parent1` contains it.

Solution:

```
const gp = document.getElementById('grandparent');
const p1 = document.getElementById('parent1');
const li = document.querySelector('li');

gp.contains(li); // true — li is a descendant
p1.contains(li); // false — li is inside parent2, not parent1
```

Q8: querySelectorAll from a Parent (Scoped Query)

Task: Get all `.active` items but ONLY inside `#parent2`.

Solution:

```
const p2 = document.getElementById('parent2');
p2.querySelectorAll('.active'); // NodeList [li.active, li.active]

// vs document-level which searches everywhere
document.querySelectorAll('.active'); // same here, but scope matters in bigger D
```

Q9: Tricky — parentElement Chain with Condition

Task: From `#child2`, walk up ancestors until you find one with class `container`. Do it WITHOUT `closest()`.

Solution:

```
function findAncestorByClass(el, className) {
  let current = el.parentElement;
  while (current) {
    if (current.classList.contains(className)) return current;
    current = current.parentElement;
  }
  return null;
}

findAncestorByClass(document.getElementById('child2'), 'container');
// #grandparent - this is essentially what closest() does internally
```

Q10: Sibling Collector

Task: Get ALL siblings of `#child2` (not including itself).

Solution:

```
const c2 = document.getElementById('child2');
const siblings = [...c2.parentElement.children].filter(el => el !== c2);
// [p#child1, span#child3]
```

Quick Reference Cheatsheet

Direction	Element Only	All Nodes
Parent	parentElement	parentNode
Children	children	childNodes
First Child	firstElementChild	firstChild
Last Child	lastElementChild	lastChild
Prev Sibling	previousElementSibling	previousSibling
Next Sibling	nextElementSibling	nextSibling
Ancestor Search	closest(selector)	manual loop
Contains	contains(node)	—

Interview Tip: The `Element` vs `Node` distinction trips up most candidates. `Element` versions skip text/comment nodes, `Node` versions include everything.